

Apostila de NumPy

Teoria e exemplos práticos em Python

Computação numérica eficiente com arrays

1. Introdução ao NumPy

O **NumPy** (Numerical Python) é a biblioteca fundamental para computação científica em Python. Ele fornece o objeto **ndarray** — um array N-dimensional homogêneo — além de funções matemáticas otimizadas, álgebra linear, geração de números aleatórios e ferramentas para integração com C/C++ e Fortran.

As operações em NumPy são **vetorizadas**: executadas em código C compilado, o que torna o cálculo centenas de vezes mais rápido do que loops puros em Python. É a base de pandas, scikit-learn, TensorFlow, PyTorch e SciPy.

Por que usar NumPy?

- **Velocidade**: operações vetorizadas em C.
- **Memória eficiente**: arrays homogêneos ocupam menos espaço que listas.
- **Sintaxe expressiva**: broadcasting e indexação avançada.
- **Ecossistema**: padrão de fato para dados numéricos em Python.

2. Instalação

```
pip install numpy
```

```
# verificar versão  
python -c "import numpy as np; print(np.__version__)"
```

Convenção quase universal: **import numpy as np**. Usaremos esse alias em todos os exemplos.

3. O objeto ndarray

O **ndarray** é uma grade N-dimensional de elementos do **mesmo tipo** (dtype). Diferente das listas Python, ele tem tamanho fixo na criação e tipo único, permitindo armazenamento contíguo na memória.

```
import numpy as np

a = np.array([1, 2, 3, 4, 5])
print(a)           # [1 2 3 4 5]
print(type(a))     # <class 'numpy.ndarray'>
print(a.dtype)     # int64
print(a.shape)     # (5,)
print(a.ndim)      # 1
print(a.size)      # 5
print(a.itemsize)  # 8 bytes por elemento
```

Atributos principais: **shape** (dimensões), **ndim** (nº de eixos), **size** (total de elementos), **dtype** (tipo) e **itemsize** (bytes por elemento).

Arrays 2D e 3D

```
m = np.array([[1, 2, 3],
              [4, 5, 6]])
print(m.shape)  # (2, 3) -> 2 linhas, 3 colunas
print(m.ndim)   # 2

t = np.zeros((2, 3, 4))  # tensor 3D
print(t.shape)  # (2, 3, 4)
```

4. Criando arrays

Existem várias formas de criar arrays. Escolha a que melhor expressa sua intenção:

```
np.array([1,2,3])      # a partir de lista
np.zeros((3,4))        # tudo zeros
np.ones((2,3))         # tudo uns
np.full((2,2), 7)      # preenchido com 7
np.eye(4)              # identidade 4x4
np.arange(0, 10, 2)     # [0 2 4 6 8] - como range
np.linspace(0, 1, 5)    # 5 pontos igualmente espaçados [0, .25, .5, .75, 1]
np.random.rand(3, 2)    # aleatórios uniforme [0,1)
np.random.randn(3)      # normal padrão
np.empty((2,2))         # sem inicializar (lixo de memória)
```

arange usa passo (como range); **linspace** usa número de pontos. Para grades regulares, **linspace** é geralmente mais seguro (evita problemas de ponto flutuante na fronteira).

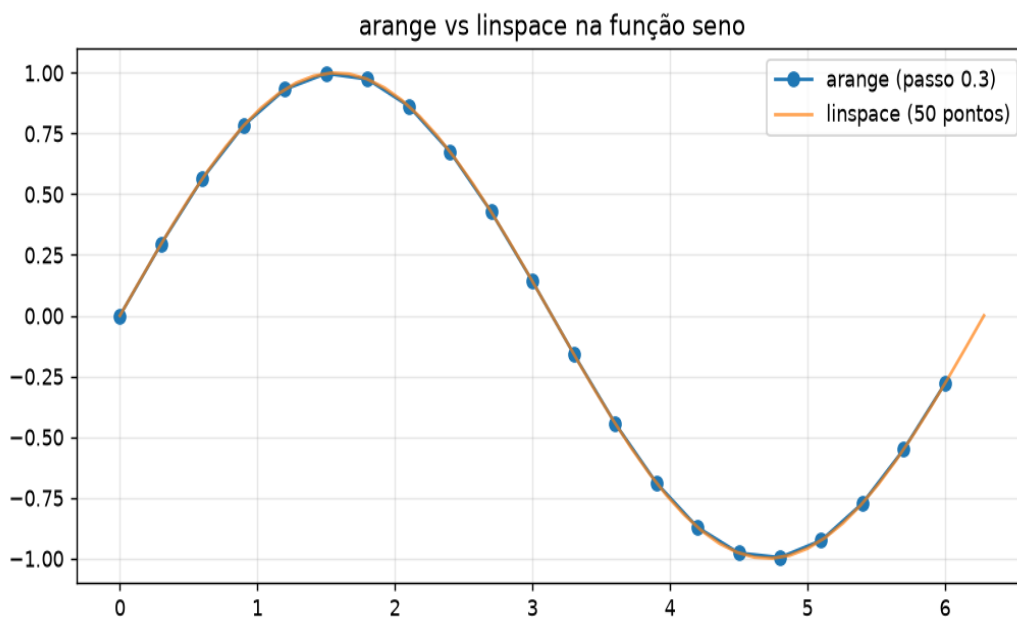


Fig. 1 — arange dá passos discretos; linspace garante N pontos exatos.

5. Tipos de dados (dtype)

NumPy oferece tipos numéricos precisos: **int8**, **int16**, **int32**, **int64**, **uint8**..**uint64**, **float16**, **float32**, **float64**, **complex64/128**, **bool**, etc. Escolher o dtype certo economiza memória e acelera o cálculo.

```
a = np.array([1, 2, 3], dtype=np.float32)
print(a.dtype)           # float32

b = a.astype(np.int16)    # conversão de tipo
print(b.dtype, b)        # int16 [1 2 3]

# inferência automática
np.array([1, 2, 3.0]).dtype # float64 (promove para o tipo mais geral)
```

6. Indexação e slicing

Funciona como em listas Python, mas estendido a N dimensões. Slices em NumPy retornam **views** (referências), não cópias — alterar a view altera o original.

```
a = np.arange(12).reshape(3, 4)
# [[ 0  1  2  3]
#   [ 4  5  6  7]
#   [ 8  9 10 11]]

a[0, 0]    # 0 (linha 0, coluna 0)
a[1]       # array([4, 5, 6, 7])
a[:, 1]    # array([1, 5, 9]) - coluna 1
a[1:, 2:]  # bloco inferior-direito
a[-1, -1]  # 11

# atenção: slice é VIEW
sub = a[0]
sub[0] = 999
print(a[0,0]) # 999 (modificou o original!)
# use .copy() se quiser cópia independente:
sub = a[0].copy()
```

Indexação booleana e fancy indexing

```
x = np.array([10, 20, 30, 40, 50])

# máscara booleana
x[x > 25]          # array([30, 40, 50])
x[(x > 15) & (x < 45)] # array([20, 30, 40])

# fancy indexing (lista de índices)
x[[0, 2, 4]]       # array([10, 30, 50])

# atribuição condicional
x[x > 25] = 0
print(x)           # [10 20  0  0  0]
```

Use **&**, **|**, **~** (não **and/or/not**) com máscaras booleanas, e sempre coloque cada condição entre parênteses.

7. Reshape, transposição e concatenação

```
a = np.arange(12)
b = a.reshape(3, 4)      # 3 linhas, 4 colunas
c = a.reshape(2, -1)     # -1 = NumPy infere -> (2, 6)

b.T                      # transposta -> (4, 3)
b.ravel()                # achata para 1D (view se possível)
b.flatten()              # achata para 1D (sempre cópia)

x = np.array([1,2,3]); y = np.array([4,5,6])
np.concatenate([x, y])   # [1 2 3 4 5 6]
np.vstack([x, y])        # empilha verticalmente -> shape (2,3)
np.hstack([x, y])        # horizontal -> [1 2 3 4 5 6]
np.stack([x, y], axis=1)  # novo eixo -> [[1,4],[2,5],[3,6]]
```

8. Operações vetorizadas

A grande vantagem do NumPy: operações elemento a elemento sem loops explícitos.

```
a = np.array([1, 2, 3, 4])
b = np.array([10, 20, 30, 40])

a + b      # [11 22 33 44]
a * b      # [10 40 90 160]
a ** 2     # [1 4 9 16]
np.sqrt(b) # raiz quadrada elemento a elemento
np.exp(a)  # exponencial
np.log(b)  # logaritmo natural

# comparação retorna array booleano
a > 2      # [False False True True]
```

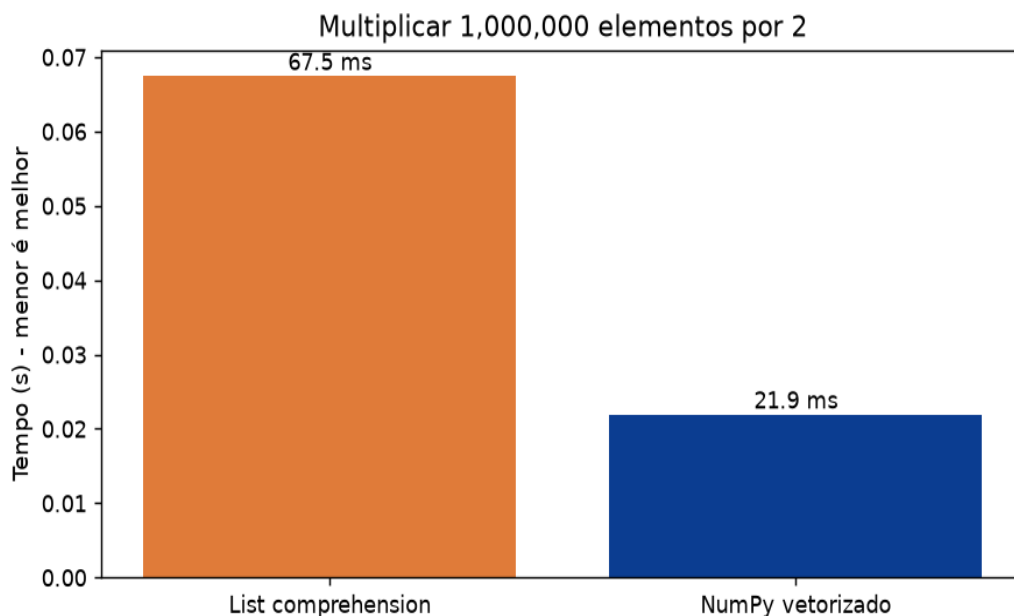


Fig. 2 — Vetorização do NumPy vs. loop Python puro.

9. Broadcasting

Broadcasting é o conjunto de regras que permite operações entre arrays de **shapes diferentes**. NumPy estende virtualmente as dimensões menores para que coincidam, sem cópia real de dados.

Regras: comparando shapes da direita para a esquerda, as dimensões são compatíveis quando são iguais ou uma delas é 1.

```
# escalar com array
np.array([1,2,3]) + 10      # [11 12 13]

# vetor (3,) com matriz (4,3) - alinha pela direita
M = np.ones((4, 3))
v = np.array([1, 2, 3])      # shape (3,)
M + v                        # broadcast -> cada linha recebe v

# coluna (4,1) com linha (1,3) -> matriz (4,3)
col = np.array([[1],[2],[3],[4]]) # (4,1)
row = np.array([[10, 20, 30]])    # (1,3)
col + row      # (4,3): tabela de soma
```

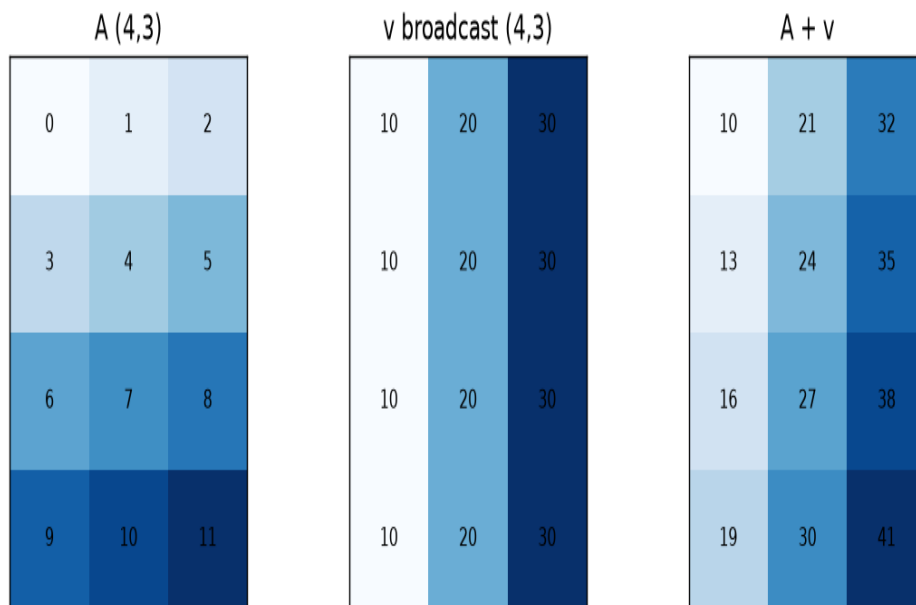


Fig. 3 — O vetor v (shape $(3,)$) é estendido virtualmente para somar a A .

10. Funções de agregação

Reduzem um array a um escalar (ou a um array de menor dimensão usando **axis**).

```
x = np.array([[1, 2, 3],
              [4, 5, 6]])

x.sum()          # 21
x.mean()         # 3.5
x.min(), x.max() # (1, 6)
x.std()          # desvio-padrão
x.argmax()       # índice do máximo (em array achatado)

# eixo: 0 percorre linhas (resultado por coluna)
x.sum(axis=0)    # [5 7 9] - soma de cada coluna
x.sum(axis=1)    # [6 15] - soma de cada linha

# cumulativos
np.cumsum([1,2,3,4]) # [1 3 6 10]
```

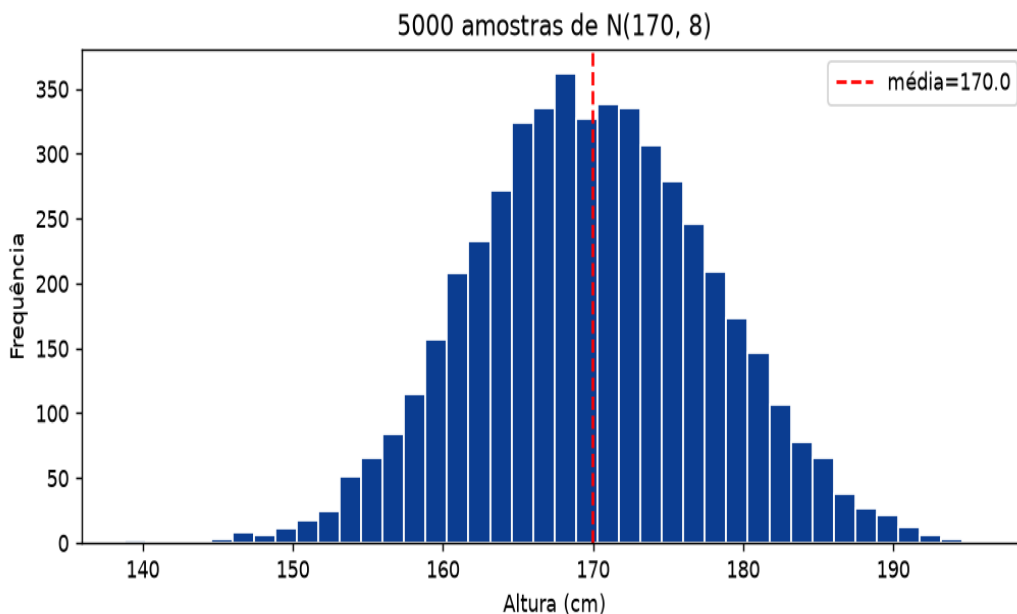
Regra prática para **axis**: ele indica o eixo que **some/colapsa**. **axis=0** elimina o eixo das linhas, deixando um resultado por coluna; **axis=1** elimina o eixo das colunas, deixando um por linha.

11. Números aleatórios

```
rng = np.random.default_rng(seed=42) # gerador moderno (recomendado)

rng.random(5)          # uniforme [0,1)
rng.integers(1, 7, size=10) # 10 dados de 6 faces
rng.normal(loc=0, scale=1, size=1000) # normal N(0,1)
rng.choice(['A', 'B', 'C'], size=5, p=[.5, .3, .2])
rng.shuffle(arr)        # embaralha no lugar
```

Use sempre **default_rng()** em código novo. A interface antiga `np.random.rand/randn` ainda funciona, mas o gerador moderno é mais rápido, com melhor estatística e thread-safe.



12. Álgebra linear

```
A = np.array([[1, 2],
              [3, 4]])
B = np.array([[5, 6],
              [7, 8]])

A * B          # produto ELEMENTO A ELEMENTO
A @ B          # produto MATRICIAL (== np.matmul(A,B))
A.dot(B)       # equivalente

np.linalg.inv(A)      # inversa
np.linalg.det(A)      # determinante
np.linalg.eig(A)      # autovalores e autovetores

# resolver sistema linear Ax = b
b = np.array([5, 11])
x = np.linalg.solve(A, b)
print(x)           # solução exata, mais estável que inv(A) @ b
```

■ * é multiplicação elemento a elemento; @ é produto matricial. Misturar os dois é um erro frequente para quem vem do MATLAB.

13. Seleção condicional: where, select, clip

```
x = np.array([-2, -1, 0, 1, 2])

np.where(x > 0, x, 0)      # ReLU: [0 0 0 1 2]
np.where(x > 0)            # ((array([3,4]),)) - índices onde True

np.clip(x, -1, 1)         # limita ao intervalo [-1, 1] -> [-1 -1 0 1 1]

# np.select para múltiplas condições
cond = [x < 0, x == 0, x > 0]
val = ["neg", "zero", "pos"]
np.select(cond, val)      # ['neg' 'neg' 'zero' 'pos' 'pos']
```

14. Entrada e saída

```
# binário rápido (formato .npy)
np.save("dados.npy", arr)
arr2 = np.load("dados.npy")

# múltiplos arrays
np.savez("pacote.npz", a=arr1, b=arr2)
d = np.load("pacote.npz"); d["a"], d["b"]

# texto (CSV)
np.savetxt("dados.csv", arr, delimiter=",", fmt="%.3f")
np.loadtxt("dados.csv", delimiter=",")

# CSV com mistura de tipos / cabeçalho
np.genfromtxt("dados.csv", delimiter=",", names=True, dtype=None)
```

Para dados tabulares heterogêneos, prefira **pandas**. NumPy é ideal para arrays numéricos puros.

15. Exemplo prático: análise de notas

Vamos juntar várias ideias em um caso real: dada uma matriz de notas (alunos × disciplinas), calcular estatísticas, normalizar e identificar destaques.

```
rng = np.random.default_rng(7)
notas = rng.integers(0, 11, size=(20, 4)) # 20 alunos, 4 disciplinas
disciplinas = np.array(['Mat', 'Fis', 'Quim', 'Bio'])

# média por aluno e por disciplina
media_aluno = notas.mean(axis=1)
media_disciplina = notas.mean(axis=0)

# aprovados (média >= 6)
aprovados = media_aluno >= 6
taxa = aprovados.mean() * 100

# melhor aluno em cada disciplina
melhor_idx = notas.argmax(axis=0)

# normalizar 0-1 por disciplina (broadcasting!)
minimo = notas.min(axis=0)
maximo = notas.max(axis=0)
norm = (notas - minimo) / (maximo - minimo)

print(f"Taxa de aprovação: {taxa:.1f}%")
print("Média por disciplina:", dict(zip(disciplinas, media_disciplina.round(2))))
```

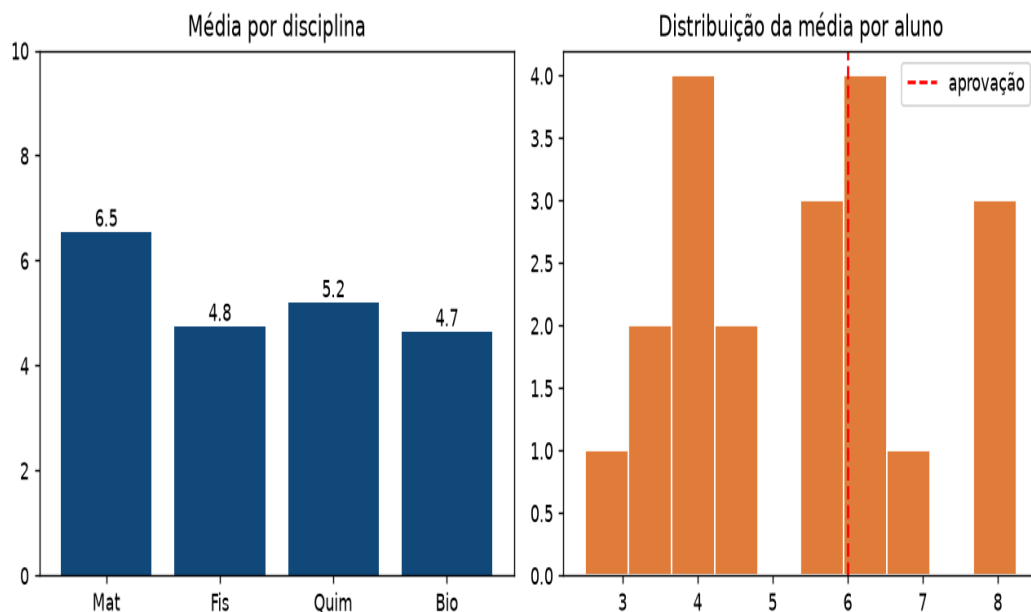


Fig. 5 — Estatísticas calculadas com agregações ao longo dos eixos.

16. Boas práticas

- **Pense em arrays, não em loops:** se você está escrevendo *for*, pergunte se há uma operação vetorizada equivalente.
- **Pré-aloque** arrays grandes com `np.zeros` em vez de fazer `append` repetido (que recria o array).
- **Cuidado com views:** slices compartilham memória; use `.copy()` quando precisar de independência.
- **Use o dtype mínimo** que atende à precisão necessária (float32 ocupa metade de float64).
- **Defina seed** em código reprodutível com `np.random.default_rng(seed)`.

- Prefira `@` a `np.dot` para multiplicação matricial — mais legível.
- Para dados rotulados (linhas, colunas com nomes), suba para **pandas**; NumPy é a fundação.

Próximos passos

- **pandas** — análise de dados tabulares (construído sobre NumPy).
- **SciPy** — otimização, estatística avançada, integração numérica.
- **scikit-learn** — machine learning clássico.
- **matplotlib / seaborn** — visualização.
- Documentação oficial: numpy.org/doc/stable/